

AutoTrade Project Report

Autonomous Trading Pipeline

Software Engineering Project

Table of Contents

- Executive Summary
- 1. Introduction
 - 1.1 Purpose and Scope
 - 1.2 Product Overview
 - 1.3 Structure of the Document
 - 1.4 Terms, Acronyms, and Abbreviations
- 1. Project Management Plan
 - 2.1 Project Organization
 - 2.2 Lifecycle Model Used
 - 2.3 Risk Analysis and Mitigation Strategies
 - 2.4 Hardware and Software Resource Requirements
 - 2.5 Deliverables and Schedule
 - 2.6 Monitoring, Reporting, and Controlling Mechanisms
 - 2.7 Professional Standards
 - 2.8 Configuration Management
 - 2.9 Impact on Individuals and Organizations
- 1. Requirement Specifications
 - 3.1 Stakeholders
 - 3.2 Use Case Model
 - 3.3 Rationale for Use Case Model
 - 3.4 Detailed Functional and Non-Functional Requirements
- 1. Architecture
 - 4.1 Architectural Style Used
 - 4.2 Architectural Model
 - 4.3 Technology, Software, and Hardware Used
 - 4.4 Rationale for Architectural Style
- 1. Design
 - 5.1 User Interface Design
 - 5.2 Static Database and State Model
 - 5.3 Dynamic Model and Execution Sequence
 - 5.4 Rationale for Detailed Design Choices
 - 5.5 Traceability
- 1. Test Plan
 - 6.1 System Level Test Cases
 - 6.2 Traceability to Use Cases
 - 6.3 Test Generation Techniques
 - 6.4 Assessment of Test Suite
- Acknowledgment
- References

SE 4485 Software Engineering Project: AutoTrade

Final Report

Executive Summary

AutoTrade is a sophisticated, autonomous trading pipeline designed to completely automate the entire lifecycle of stock selection, news aggregation, comprehensive multi-modal financial analysis, and direct simulated trade execution. Built upon a robust backend of Python and LangGraph, the system heavily leverages state-of-the-art Large Language Models (LLMs), specifically the Google Gemini architecture (`gemma-4-31b-it`), to perform in-depth sentiment, quantitative, and algorithmic analysis on various financial assets.

The system seamlessly integrates with external data providers, such as the Finnhub API and `yfinance`, to fetch real-time market data, company profiles, financial statements, and historical price candles. It executes simulated trades against a local SQLite database using the Peewee ORM, managing a virtual portfolio complete with cash balance ledgers and individual stock holdings. By orchestrating a Directed Acyclic Graph (DAG) of specialized LLM agents, AutoTrade emulates the complex, multi-faceted decision-making process of a professional hedge fund.

1. Introduction

1.1 Purpose and Scope

The primary purpose of the AutoTrade project is to simulate an autonomous trading agent capable of making intelligent, data-driven investment decisions. These decisions are formulated based on real-time news sentiment, historical price action, and complex quantitative financial indicators.

The scope of this project encompasses: - **Data Ingestion and Processing:** Fetching real-time quotes, news, and historical data.

- **Multi-Agent Parallel Analysis:** Utilizing LLM-powered agents to independently assess stocks from algorithmic, sentiment, and quantitative perspectives.

- **Portfolio State Management:** Synthesizing independent analyses to make cohesive trading decisions while respecting risk management boundaries (e.g., stop-loss, take-profit).

- **Trade Execution:** Simulating the execution of trades and maintaining a persistent, transactional ledger of the portfolio.

- **Reporting:** Generating transparent, human-readable PDF reports that detail the reasoning behind every automated decision.

1.2 Product Overview

Core Capabilities: - **Algorithmic Stock Selection:** Automatic selection of candidate stocks based on predefined market criteria, ensuring the system continually monitors active or high-potential assets.

- **Real-Time Context Aggregation:** Aggregation and LLM-driven refinement of real-time news articles to eliminate noise and extract highly relevant market sentiment signals.

- **Tri-Modal Parallel Analysis:**

- *Algorithmic Prediction:* Analyzes moving averages, momentum, and historical trends.

- *Sentiment Analysis:* Parses news headlines and recent articles to gauge public and institutional sentiment.

- *Quantitative Analysis:* Examines fundamental financial data, quarterly earnings, and balance sheets.

- **Portfolio Synthesis:** The `Portfolio Manager` agent aggregates the verdicts (Buy/Sell/Hold) and probabilities from the three analysts, checking the current cash balance and existing portfolio holdings to ensure strict diversification and avoid over-concentration.

- **Database Ledger Management:** Execution of simulated trades with strict transactional integrity via SQLite, maintaining tables for executed trades, balance ledgers, and active portfolio assets.

- **Audit and Reporting:** Generation of detailed PDF reports utilizing the `fpdf2` library, summarizing the rationale, target prices, and stop-loss metrics behind trading decisions.

Primary User Scenarios: - **Automated Daily Pipeline:** A user configures a cron job to run `run_graph.py` daily. The pipeline autonomously researches a volatile market segment, processes hundreds of data points, and executes a verified trade recommendation without any manual intervention, updating the internal database ledger accordingly.

- **Strategy Backtesting and Verification:** A developer uses the system to test how different LLM prompts or analyst agent weights affect the long-term profitability of the simulated portfolio.

1.3 Structure of the Document

This document outlines the project management plan, detailed software requirements, architectural design, specific static and dynamic design models, and test planning strategies for the AutoTrade system. It maps the Python/LangGraph implementation to standardized software engineering reporting practices.

1.4 Terms, Acronyms, and Abbreviations

- **LLM:** Large Language Model (e.g., Google Gemini).
- **API:** Application Programming Interface.
- **DAG:** Directed Acyclic Graph (used by LangGraph).
- **ORM:** Object-Relational Mapping (e.g., Peewee).
- **CLI:** Command Line Interface.
- **SDLC:** Software Development Life Cycle.
- **JSON:** JavaScript Object Notation.

2. Project Management Plan

2.1 Project Organization

The AutoTrade source code is systematically structured into highly cohesive, loosely coupled modular components:

- `graph/`: Contains the LangGraph orchestration (`main.py`), state definitions (`state.py`), and individual agent node implementations (`nodes/` directory).
- `database/`: Houses the Peewee ORM schema models (`models.py`) and standard CRUD (Create, Read, Update, Delete) operations (`crud.py`).
- `services/`: Provides shared utility functions, retry decorators, API interactions, and PDF reporting capabilities (`reporting.py`).
- `reports/`: The output directory where generated PDF analytical summaries are saved.

2.2 Lifecycle Model Used

The development of AutoTrade followed an Agile, iterative Software Development Life Cycle (SDLC). The initial sprints focused on establishing the Peewee database schema and the foundational LangGraph state machine. Subsequent iterations introduced individual LLM analyst nodes, progressively refining the prompting strategy and implementing the Finnhub API integrations.

2.3 Risk Analysis and Mitigation Strategies

- **API Rate Limits and Network Failures:**
 - *Risk*: Hitting Finnhub or Gemini rate limits, resulting in pipeline crashes.
 - *Mitigation*: The system implements robust `@retry_on_error` decorators with exponential backoff to gracefully handle HTTP 429 and 500 errors.
- **LLM Hallucinations and Unpredictable Outputs:**
 - *Risk*: The LLM might suggest illogical trades or output malformed, non-JSON text.
 - *Mitigation*: Strict system prompting forces the LLM to output pure JSON. A dedicated `Portfolio Manager` node acts as a final sanity check, strictly enforcing risk limits (e.g., never risking more than 20% of total cash on a single trade) and applying fallback "HOLD" decisions if parsing fails.
- **State Corruption:**
 - *Risk*: Concurrent trades causing race conditions in the SQLite database.
 - *Mitigation*: Implementation of SQL transactions and strict constraints within the CRUD layer.

2.4 Hardware and Software Resource Requirements

- **Software Dependencies**: Python 3.10+, SQLite3. Key libraries include `langgraph`, `peewee`, `google-genai`, `yfinance`, `finnhub-python`, `beautifulsoup4`, and `fpdf2`.
- **Hardware Resources**: A standard modern workstation with an active broadband internet connection is sufficient. LLM inference is handled entirely off-device via the Google Gemini Cloud API.

2.5 Deliverables and Schedule

- **Milestone 1**: Core LangGraph architecture setup, `AgentState` definition, and Peewee ORM schema initialization.
- **Milestone 2**: Implementation of data ingestion nodes (`Stock Picker`, `News`

Aggregator, News Refiner).

- **Milestone 3:** Implementation of the concurrent Tri-Modal Analyst nodes (Quant, Sentiment, Algo) utilizing LLM Tool Calling.

- **Milestone 4:** Implementation of the Portfolio Manager, Execution Agent, and the persistent SQLite ledger.

- **Milestone 5:** PDF Reporting module implementation and comprehensive end-to-end integration testing (`run_graph.py`).

2.6 Monitoring, Reporting, and Controlling Mechanisms

During execution, the pipeline emits highly detailed console logs indicating node entry/exit events, real-time portfolio cash state changes, and intermediate LLM outputs. For persistent tracking, a reporting module uses `fpdf2` to dynamically generate PDF summaries of the LLM's analytical rationale, storing them in a timestamped `reports/` subdirectory.

2.7 Professional Standards

The Python codebase strictly adheres to standard PEP-8 style guidelines. Robust Python type hinting (`typing.TypedDict`, `__future__.annotations`) is utilized extensively to maintain rigorous type safety across complex dictionary-based state transitions within the LangGraph ecosystem.

2.8 Configuration Management

Version control is managed via Git. Python dependencies are strictly pinned and managed via `requirements.txt`. Sensitive credentials and environment configurations (e.g., `GOOGLE_API_KEY`, `FINNHUB_API_KEY`) are entirely isolated from the source code using a `.env` configuration file, utilizing `.env.example` as a template for new deployments.

2.9 Impact on Individuals and Organizations

AutoTrade provides individuals and small trading firms with a powerful framework to automate previously tedious, highly manual financial research. By orchestrating multiple LLMs, it democratizes access to sophisticated multi-agent quantitative analysis methodologies that have traditionally been exclusively reserved for large-scale institutional hedge funds.

3. Requirement Specifications

3.1 Stakeholders

- **Retail Investors and Traders:** Individuals looking to heavily automate their investment research and rigorously backtest autonomous trading strategies without human emotional bias.
- **Software Engineers and Researchers:** Professionals aiming to extend the multi-agent architecture for custom financial use cases, adding proprietary indicators or novel LLM prompting techniques.

3.2 Use Case Model

Use Case 1: Execute Complete Autonomous Trading Pipeline - Primary Actor: End User / System Cron Job

- **Preconditions:** The `.env` file is populated with valid API keys. The SQLite database is initialized.

- **Main Success Scenario:**

1. The user initiates the pipeline via the `python run_graph.py` CLI command.
 2. The `Stock Picker` autonomously queries market data to select an optimal candidate stock.
 3. The `News Aggregator` and `Refiner` gather and distill relevant market context.
 4. The system fans out execution to perform a parallel tri-modal analysis (Quant, Sentiment, Algo).
 5. The `Portfolio Manager` synthesizes the results, checks the database for available funds, and formulates a trading decision (BUY, SELL, or HOLD).
 6. The `Execution Agent` commits the transaction to the database ledger, updating the user's virtual equity and cash balance.
- **Postconditions:** The SQLite database reflects the new portfolio balance. A PDF report is generated outlining the analytical rationale.

3.3 Rationale for Use Case Model

The system is architecturally designed for high autonomy. Therefore, the primary functional requirement revolves around a single trigger event that cascades deterministically through multiple automated sub-tasks. This strongly minimizes the need for continuous user intervention and maximizes computational efficiency.

3.4 Detailed Functional and Non-Functional Requirements

Functional Requirements: - **FR1 (Orchestration):** The system must execute a workflow of sequential and parallel tasks using a state graph.

- **FR2 (LLM Integration):** The system must utilize the Gemini API to process textual financial data and output structured JSON decisions.

- **FR3 (Data Retrieval):** The system must fetch stock quotes, company profiles, and financial news via the Finnhub API.

- **FR4 (Persistence):** The system must maintain a transactional database containing `balance_ledger`, `portfolio`, and `executed_trades` tables.

Non-Functional Requirements: - **NFR1 (Performance):** The parallel fan-out analysis for a single stock must execute and resolve within 120 seconds, assuming normal API latency.

- **NFR2 (Reliability):** The database architecture must accurately track portfolio cash and equity without race conditions. If an LLM fails to output valid JSON, the system must default to a safe "HOLD" state rather than crashing.
- **NFR3 (Extensibility):** The LangGraph DAG architecture must be sufficiently modular to support the seamless addition of new analyst nodes (e.g., a "Macroeconomic Analyst") without fundamentally altering the core pipeline.

4. Architecture

4.1 Architectural Style Used

The AutoTrade system leverages a hybrid **Workflow/DAG Orchestration** architecture combined with a highly modular **Multi-Agent System** paradigm.

4.2 Architectural Model

LangGraph is utilized to compile and orchestrate a StateGraph. The entire workflow is formally represented as a Directed Acyclic Graph (DAG) with both sequential dependencies and parallel branches:

1. START -> Stock Picker -> News Aggregator -> News Refiner
2. **Fan-Out:** News Refiner splits execution simultaneously to:
 - Algorithmic Predictor (with dynamic routing to algo_tools)
 - Sentiment Analyst (with dynamic routing to sentiment_tools)
 - Quantitative Analyst (with dynamic routing to quant_tools)
3. **Fan-In:** The parallel nodes converge at the Portfolio Manager.
4. Portfolio Manager -> Execution Agent -> END.

4.3 Technology, Software, and Hardware Used

- **Workflow Orchestration:** LangGraph, LangChain Core.
- **Artificial Intelligence Models:** Google Gemini API (gemma-4-31b-it).
- **Financial Data Providers:** Finnhub API (primary data source), yfinance, BeautifulSoup4 (web scraping).
- **Database Architecture:** Peewee ORM mapping to a localized SQLite file (trading.db).
- **Reporting & Visualization:** FPDF2 for programmatic PDF generation.

4.4 Rationale for Architectural Style

A state machine represented as a DAG (LangGraph) perfectly encapsulates the sequential data-gathering and parallel research phases of financial analysis. The explicit fan-out execution of analysts (Quant, Sentiment, Algo) dramatically reduces total pipeline execution time by concurrently processing computationally expensive LLM inferences. Furthermore, the DAG structure allows for cyclic loops, enabling the LLM agents to recursively call predefined external tools (like fetching historical price arrays) before returning to the main decision loop.

5. Design

5.1 User Interface Design

The application operates primarily in a headless, CLI-based environment. The UI consists of meticulously formatted standard output (stdout) console logs. These logs utilize ASCII borders and clear prefixes (e.g., `[portfolio_manager]`, `[stock_picker]`) to explicitly detail node entry/exit events, intermediate LLM reasoning, portfolio state changes, and final trade summaries. For human-readable summaries, the `reports/` directory serves as an asynchronous visual interface.

5.2 Static Database and State Model

The system design relies heavily on strong data schemas: - **Global State Definition:** The `AgentState` class inherits from `typing.TypedDict`. It acts as the pipeline's global memory tape, containing arrays of messages, `current_stock`, `stock_profile`, `stock_prices`, `portfolio_decision`, and the individual analyst reports.

- **Relational Data Models (Peewee SQLite):**

- `balance_ledger`: Tracks historical `DEPOSIT`, `WITHDRAWAL`, `BUY_STOCK`, and `SELL_STOCK` transactions, maintaining an authoritative `current_total_balance`.
- `portfolio`: Stores the current active equity holdings, enforcing unique constraints on the stock ticker, tracking quantity, and average price.
- `executed_trades`: Acts as an immutable audit log of all historical executions.
- `analyzed_trades`: Stores a permanent record of the summaries generated by each individual analyst prior to execution.

5.3 Dynamic Model and Execution Sequence

The dynamic execution sequence, primarily defined in `graph/main.py` and invoked by `run_graph.py`, operates as follows:

1. `run_graph.py` verifies database integrity via `init_db()` and invokes the compiled `LangGraph`.
2. The `LangGraph` sequentially passes the globally mutating `AgentState` dictionary through the data ingestion nodes (`stock_picker`, `news_aggregator`, `news_refiner`).
3. The graph splits into 3 concurrent threads for deep analysis.
4. Each analyst branch dynamically evaluates whether to request additional data. If an LLM requests tool execution, the graph conditionally routes to the respective `ToolNode`, executes the function, and loops back to the analyst node.
5. Once all analysts yield a definitive JSON verdict, the `Portfolio Manager` triggers. It aggregates the multi-agent results, queries the SQLite database via `get_current_balance()` and `get_full_portfolio()`, applies strict diversification logic, and updates the state with a finalized trading decision.
6. The `Execution Agent` reads the decision, calculates total costs, and atomically commits the transaction to the database ledger using Peewee CRUD functions.

5.4 Rationale for Detailed Design Choices

- **ToolNode Separation:** Separating functional tools into isolated `ToolNode` instances prevents the primary analytical LLM logic from becoming entangled with raw API requests.

This modular separation of concerns ensures that state transitions remain predictable.

- **Portfolio Manager as a Gateway:** By forcing all analyst outputs to bottleneck through a dedicated `Portfolio Manager`, the system guarantees that no single rogue agent can execute a trade. The manager serves as an authoritative risk-management gateway, strictly enforcing capital allocation limits.

5.5 Traceability

Every high-level requirement maps directly to distinct modules within the codebase. The requirement for modular, multi-faceted analysis corresponds exactly to the isolated python files in the `graph/nodes/` directory. The requirement for transactional integrity maps precisely to the atomic CRUD operations defined in `database/crud.py` acting upon the schemas in `database/models.py`.

6. Test Plan

6.1 System Level Test Cases

- **TC01 - Full Pipeline Golden Path:** Execute `run_graph.py` with fully valid API keys, high network connectivity, and a well-funded virtual SQLite ledger. Verify that the system successfully traverses the entire DAG from `START` to `END`, resulting in an expected entry in the `executed_trades` table.
- **TC02 - API Failure and Graceful Degradation:** Deliberately remove or invalidate the `FINNHUB_API_KEY`. Execute the pipeline and verify that the `@retry_on_error` decorators attempt recovery, and subsequently, the nodes return graceful fallback values (e.g., zero news articles, or default stock arrays) without raising unhandled Python exceptions.
- **TC03 - State Persistence and Concurrency Constraints:** Execute consecutive 'BUY' and 'SELL' transactions. Verify via direct SQLite inspection that the `portfolio` table correctly aggregates share quantities and recalculates the `average_price`, and that the `balance_ledger` accurately reflects the cash delta.
- **TC04 - Risk Management Enforcement:** Inject a mocked `AgentState` where the `Portfolio Manager` attempts to purchase shares exceeding the total available cash balance. Verify that the `Execution Agent` correctly identifies the insufficient funds and entirely rejects the transaction, preventing negative balances.

6.2 Traceability to Use Cases

- TC01 directly validates the primary "Execute Complete Autonomous Trading Pipeline" use case, ensuring the fundamental architecture operates cohesively.
- TC02 validates the "Reliability" non-functional requirement.
- TC03 and TC04 validate the integrity of the transactional database, mapping directly to FR4 (Persistence).

6.3 Test Generation Techniques

Test scenarios are derived utilizing standard Boundary Value Analysis and Equivalence Partitioning, specifically applied to financial constraints. For example, boundary tests examine system behavior when the cash balance is exactly \$0.00, or when a calculated trade quantity attempts to exceed the 20% total cash allocation limit hardcoded into the `Portfolio Manager` prompt.

6.4 Assessment of Test Suite

The objective goodness and efficacy of the test suite are evaluated by extensive functional execution coverage. The testing strategy heavily focuses on validating the conditional routing logic embedded within `graph/main.py`, ensuring that all parallel branches, potential tool-calling cyclic loops, and subsequent fan-in synchronizations execute flawlessly under a variety of network conditions and API response states.

Acknowledgment

Sincere acknowledgments and gratitude are extended to the open-source communities responsible for maintaining LangGraph, LangChain, and Peewee. Their comprehensive documentation and robust frameworks provided the essential foundational bedrock necessary to rapidly iterate and deploy this complex, agentic architecture.

References

1. *LangGraph API Reference and Documentation*. Available at the official LangChain documentation portal.
2. *Google Gemini API SDK Documentation*. Focusing specifically on `GenerateContentConfig` and structural JSON constraints.
3. *Finnhub Real-Time API Documentation*. Referencing the websocket streams and REST endpoints for company news and fundamental financial data.
4. *Peewee Lightweight Python ORM Documentation*. Detailing schema migrations and SQLite constraint enforcement.